课径 CoursePath · 技术文档

> 第九届传智杯 · 全国高校 Vibe Coding 开发挑战赛 · B 组（个人赛）
>
> 本文档按评分表组织： **第 1 章 → 解题有效性与实用价值（25%）· 第 2 章 → 工程质量与架构设计（15%）· 第 3 章 → 方法创新性与意图控制（20%）· 第 4 章 → 过程可复现性与演进链路（25%）· 第 5 章是工程质量的实证（测试、缺陷、体积）· 第 6 章是主动划出的边界。**

---

0. 摘要

课径做的事：把学校那些”官网有、但没人看得懂”的官方文件（培养方案 / 转专业管理办法）， 变成结构化、可视化、能自己动手算的数据。

为什么是这件事：三份相互独立的证据指向同一个结论—— 挡住学生的不是能力，是**信息到不了手。**规则客观存在且公开，但取用成本高到让人放弃， 于是学生只能靠学长学姐的口语经验拼图，而拼出来的答案是互相矛盾的。

技术上最值得说的三点：

1. **引擎层零第三方依赖。** 领域模型、约束引擎、规划器、数据层只 `import` 标准库， 于是 `tools/validate_programs_json.py` 与 `tools/validate_rules_json.py` 能在 **不装 FastAPI、不开 venv** 的情况下跑通数据校验——这是一条独立的复现链， 而不是”先装完 web 栈再说”。
2. **能被证伪的流式。** mock 模式下的流式输出与非流式输出**逐字节相同**（有单测锁住这条等式）， 因此”这是真流式不是前端动画”不需要靠嘴说——一条 `curl -N` 就能证明。
3. **“查不到”也是要交付的数据。** 转专业规则里，校级通知没规定的类别不会被编一个数字补上， 而是以 `gaps` 显式返回，并附反向断言（`absent_keywords`）—— “校级通知里没有 70 分这个数字”这句话，脚本可以复核。

---

1. 问题与定位

1.1 痛点证据链：三类信源，各自能证明什么

调研刻意用了三类不同性质的信源，并**明确区分各自能证明什么**——这是后来所有结论的地基。

信源	性质	能证明什么	不能证明什么
校园社区公开帖（3 条，含 1 条浏览 822 次）	一手、自然发生	痛点**真实存在**，且未被提问方式引导	提问者身份不明，样本小
公开报道《我们找来 8 位成功转专业的朋友》（少数派，2021-07）	二手	**机制**：门槛如何挡人、时间窗口多窄	不能验证痛点强度（8 人中仅 2 人走本科转专业路径）
枣庄学院官方转专业文件（通知 + 3 个附件）	一手、制度	**规则怎么定**，以及规则为何难以取用	不含学生的主观体验

> 诚实说明：原计划的”访谈 2–3 位转专业同学”**未能执行**——没有可接触的转专业学生。 因此证据链改由公开社区数据承担，并在第 6 章显式声明这一限制。

1.2 三条核心发现

发现一：痛点在「信息可达性」，不在「信息缺失」。

社区帖里最值钱的一句话是：

> **「你看看官网　官网有详细介绍」**

它证明了官方信息是公开的、完整的——但学生不会去翻。同一件事（空乘专业能否转出） 在一条帖子下得到了三个互相矛盾的答案（“好像不能转” / “艺术类只能在艺术类里转” / “看官网”）。 **矛盾本身就是证据**：学生无法自行判断，只能赌。

后来从校方文件里找到了这个矛盾的**制度性原因**（见发现三）。

发现二：准入门槛被系统性高估。

社区帖里学生焦虑”通识课是不是要卡 70 分”，而回复是”70 分很简单的，有军训军事理论给你往上拉分”。 **门槛本身不难，难的是不知道门槛是什么。**

这条焦虑后来被产品闭环掉了：把校级通知结构化之后，“70 分”这个数字在原文里 **一次都没有出现**（`gaps` 里的反向断言 `absent_keywords`，校验脚本可复核）。 答案要么在 21 个学院的方案里，要么根本不存在——但至少学生不用再猜。

二手报道提供了同构的佐证（相隔四年的独立信源）：

> 虽然转专业考试本身的难易程度可能并不会太高……「应该可以说是**完全没准备**」， 但**门槛设置就足够把大批人马挡在外面**

发现三：官方文件本身就难以取用。

校方通知里埋着答案：具体门槛**分散在 21 个学院各自网站公示的方案里**，校级通知只给了框架。 学生要拼出完整答案，得跨：

```
《普通高等学校学生管理规定》（教育部令第41号）          ← 未获取
  └─《枣庄学院转专业管理暂行办法（试行）》〔2017〕165号   ← 未获取
      └─ 本通知（2024级）                                ← 已获取
          ├─ 21 个学院的转专业工作方案                    ← 未获取，散在 21 个网站
          └─ 学校学分认定管理办法                         ← 未获取，被通知"甩"出去
              └─ 各专业培养方案                           ← 未获取，产品最终要吃的数据
```

4~5 份文件、21 个网站。 这就是”信息到不了手”的结构性原因。

1.3 学校自己的文件，就是产品的需求书

通知「工作要求」第 2 条原文：

> 学生应**充分了解所申请专业的相关情况**，根据自己的志趣和能力慎重选择，不可盲目跟从， 以免转专业后造成学习不适应，影响学业。

官方承认了三件事：学生**需要**了解专业、**需要**做规划、**不了解会有后果。** 而校方提供的手段，只有这一句口号。

课径做的正是学校要学生做、却没有给工具的那件事。

1.4 定位：双入口，一套引擎

调研显示痛点分布在”转专业前”和”转专业后”两端，因此定位为双入口—— 但两个入口**共用同一套核心引擎**，因为两类输入是同一种东西：**官方文件。**

入口	学生的原话	对应能力
认知入口（转专业前）	“还有其他特殊要求吗” “跨校区转有限制吗”	规则查询（`/api/rules` + 前端「转专业规则」视图，含”查不到”清单）
核算入口（转专业后）	“学分折抵……自己核算”	培养方案可视化 + 冲突检测 + **补修/重修清单**

按赛题的十个推荐选题方向，本项目属于**方向 5「AI 驱动的高交互教育与可视化」**（把抽象的培养方案变成可拖拽、可筛选的可视化面板）， 并以**方向 2「场景化轻量 RAG 知识库」**同款的真实信息问题作为内容支撑。 这个组合不是拼凑：方向 5 最容易在”实用价值”上被判成玩具，而真实的官方文件与调研 恰好补上了那一块。

---

2. 架构设计

2.1 分层与依赖铁律

```
┌─ 边缘层 ──────────────────────────────────────────┐
│  schemas.py（Pydantic 请求体）   main.py（FastAPI 路由）│  ← 允许 fastapi / pydantic
└──────────────────────────────────────────────────┘

┌─ 引擎层 ──────────────────────────────────────────┐
│  constraints.py（约束引擎，纯函数）                  │
│  planner.py（路径规划）     llm.py（模型适配）        │  ← 仅标准库
│  sse.py（SSE 帧格式）                              │
└──────────────────────────────────────────────────┘

┌─ 领域与数据层 ────────────────────────────────────┐
│  models.py（领域模型，纯 dataclass，无 I/O）         │
│  store.py（唯一的文件 I/O）   config.py（路径基准）    │  ← 仅标准库
└──────────────────────────────────────────────────┘
```

这条分工不是洁癖，它换来一个具体能力：

```
py -3.11 tools/validate_programs_json.py    # 不装 fastapi、不开 venv 就能跑
```

因为领域层不依赖 web 栈，数据校验脚本可以直接复用 `app.models.from_dict` —— **同一条数据管道，既被服务运行时使用，也能被独立复现。**如果领域模型依赖 Pydantic， 这条链就断了：复现门槛从”有 Python”上升到”装完 web 栈”。

配套两条禁令： 1. **不为 `Program` / `Course` 定义 Pydantic 镜像**——两套领域模型必然漂移 2. **`models.py` 里不出现 `open()` / `json.load()`**——文件 I/O 全部收敛在 `store.py`

2.2 数据流

```
docs/调研原始材料/校方文件/*.xlsx     ← 官方原件（可追溯）
        │  tools/build_majors_json.py（可复现的转换脚本）
        ▼
data/majors-枣庄学院-2024级.json      ← 21 学院 / 108 专业 / 2 校区

docs/调研原始材料/校方文件/*.txt      ← 官方原件（转专业通知正文）
        │  tools/build_rules_json.py（人工分类 + 引用回溯）
        ▼
data/rules/*.json                     ← 转专业规则（条款 + 时间线 + 缺口）
        ┆
        └─ tools/validate_rules_json.py（零依赖自检，旁路校验，不产出数据）

data/programs/*.json                  ← 培养方案（当前为标注过的样例）
        │  app/store.py
        ▼
   领域对象 → constraints.py（纯函数）→ 冲突列表 → API / 前端
```

两个转换脚本彼此独立，但**共用同一条纪律**：产物必须能倒推回官方原件。 `build_rules_json.py` 比 `build_majors_json.py` 更进一步——它在写盘前会把每条引用 拿回原文逐字比对，对不上就拒绝生成。

每个 JSON 都配一个脚本、都带 `source` 溯源块（学校 / 原始文件名 / 生成脚本）， **从产物可以一路倒推回官方原件。**

2.3 关键设计决策

ADR-1 • 约束引擎的签名钉死为 `(plan, program) -> list[Conflict]`

这是 `CLAUDE.md` 里写死的契约，它带来两个后果：

- **已修课程必须放进 `Plan` 而不是另开参数。** 因为”已修课程满足先修” “已修课程计入学分” 这些判断本身就是约束的一部分，拆出去会让签名漂移。
- **不能用”第二个返回值”传递额外信息。** 时间冲突检测在没有排课数据时想告诉前端”我没检测”， 只能**发一条 `info` 级冲突**，不能加返回值——否则违约。

ADR-2 • 冲突报告遵守「一条缺陷只由一个检查负责」

16 种 `kind` 被划分给 6 个检查函数，**结构类缺陷统一由 `check_plan_integrity` 报出**， 其余检查遇到这些情况一律跳过。

理由：若”未知课程代码”由每个检查各自处理，同一处笔误会在 3 个检查里各报一次， 冲突列表就从”可预测”变成”靠猜” 。**确定性是可以设计出来的，不是撞出来的。**

ADR-3 • 输出确定性

```
KIND_ORDER = ("program_mismatch", "unknown_course", "duplicate_course", ...)
SEVERITY_RANK = {"error": 0, "warning": 1, "info": 2}
```

所有检查内部按 `(term, course_code)` 排序后遍历，不依赖输入顺序；最终统一排序。 两个单测专门锁死这件事：**打乱输入顺序，输出必须逐字段相等。**

ADR-4 • `planner` 返回 `PlanSuggestion` 而不是 `list[PlanItem]`

排不下课的时候（先修成环 / 学期容量不足），返回纯列表会**静默丢掉**那些课—— 前端看到一份”完整计划”却缺课。 `PlanSuggestion` 带显式的 `unscheduled` 与 `notes`， **让失败可见。**

这条与本项目对抗的是一类特定缺陷：**静默假阴性。**它比报错危险得多—— 报错会让人去查，假阴性会让人以为没问题。

ADR-5 • 前端主视图用学期泳道板，不引 React Flow

培养方案是「学期 × 课程」的**二维表**，不是需要自由布点的有向图编辑场景。 先修关系用冲突提示表达已经足够。引 React Flow 只会多一份重依赖， 且它擅长的是”节点自由布局”，而我们的约束恰恰是”课程必须落在学期列里”—— 语义上就不匹配。

拖拽用 `@dnd-kit` 而非原生 HTML5 DnD，理由是具体的： 原生 DnD **触屏完全不支持**； `dragImage` 样式不可控； `dragenter` / `dragleave` 在子元素间反复冒泡需要计数器 hack， 会让”拖拽中实时高亮合法/非法目标”**闪烁**； `dragover` 阶段读不到被拖课程的完整信息， 无法做合法性预判。

ADR-6 • 初始态是”后端算出的可行解”，不是空规划

样例培养方案不含上课时段，导致**空规划会一次报出 5 条学分缺口**（四个类别 + 总学分）。 若从空状态开始，用户打开页面看到的就是满屏红。

因此：前端以 `GET /api/programs/{id}/sample-plan` 的返回作为 reducer 的初始状态与「恢复」基线， 且 `items.length === 0` 时**不自动发起校验**（但保留手动入口，不隐藏后端的真实结果）。

演示叙事因此变成：**打开就是干净的 → 拖错一门 → 立刻冒红 → 拖回去 → 红消失。**

ADR-7 • 数据缺口用”诚实标注”处理，不编数据

时间冲突检测所需的上课时段来自**排课表**，不在培养方案里，我们没有该校真实排课数据。 处理方式是：

- `TimeSlot` 字段与重叠判定逻辑**完整实现**（含周次，因为中国高校排课的最小单元是 `(星期, 起始节, 结束节, 周次)`，只比星期+节次会在接入真实数据后产生假阳性）
- **样例数据里不填时段**——不伪造校方排课数据
- 该逻辑由单测 fixture 覆盖

- 每次校验返回**一条** `info` 级说明，明确区分”检测了但没有数据”与”检测了且无冲突”

ADR-8 • 规则数据：能引用的逐字引用，引用不了的显式列缺口

转专业规则的数据模型不是”一张字段齐全的表”，而是三类并列的结构：

```
provisions[]  可检索的条款：类别 · 判定 · 原文条号 · 逐字引用
gaps[]        查不到的类别：状态 · 说明 · 该去哪里查 · 反向断言
timeline[]    时间节点：截止日期 · 原文引用
```

理由来自一个具体事实：校级通知**只回答了四类问题中的两类。**分数门槛、考核的具体形式、 校区限制都不在这份文件里——它们散在 21 个学院的方案里，而我们没有采集到。

两条可选路线：按常识补一个”通识课 70 分”上去（演示更好看），或者如实标缺口。 选了后者，并把它做成**可复核的负向断言**：

- 每条缺口带 `absent_keywords`（门槛 / 分数线 / 绩点 / 70）
- 生成脚本与自检脚本**各写一份**空白规范化逻辑（刻意的重复：共用会让同一处错误 在两边同时成立，校验就成了自我确认），反向检索这些词，出现即失败
- 每条条款带 `clause`（原文条号）与 `quote`（逐字引用），同样回原文逐字比对

于是”我们没找到”不是一句托词，而是一句机器可以证伪的话。 这条决定了产品的边界叙事：**课径不承诺回答所有问题，但承诺不编。**

ADR-9 • 学分认定返回五类清单，每类带置信度，且学分必须守恒

`analyze_credits` 返回五类，而不是”还需要修 N 门课”：

类别	置信度	学生要做什么
已认定	高	不用管（课号相同且已通过）
需重修	高	走重修（课号相同但未通过）
可能可折抵	**中**	**带成绩单去教务处确认**（课名、学分都对得上，课号不同）
需补修	高	按培养方案选课（成绩单里对不上）
待认定	低	原专业独有课程，由学院判断能否计入

合并成”还差 N 门”看起来更简洁，实际把”去选课”和”去问教务处”混成同一件事。 这与本项目对抗静默假阴性的取向一致：**能区分的状态就要区分开。**

为什么需要「可能可折抵」这一类。 跨专业转学时，公共课（高数、英语、体育、思政） 在原专业往往有另一套课号。如果只按课号匹配，它们会全部落进「需补修」—— 学生会被告知”你要重修 11 门公共课”，这正是评审意见里担心的 「功能不完善导致学生没拿到学分」。因此加一层**确定性**匹配： 课名归一

化（全角转半角、去空白与标点、**不删括号里的序号**）后相同，且学分相同， 即归入「可能可折抵」。我们仍然**不判断能不能折抵**——那需要学校的认定办法—— 只把它单独列出来，动作是”去确认”，不并进已认定。

守恒是硬约束。 前四类（不含 `pending`）的学分之和恒等于培养方案总学分： 每门方案课必须恰好落在一类里，不重不漏。 `pending` 是成绩单里不属于本方案的课， 单独统计、不参与守恒。这条断言已写进单测（`test_credit_conservation_across_buckets` 与 `test_every_program_course_lands_in_exactly_one_bucket`）， 它挡的正是”同一门课同时算进需补修和待认定”这类重复计算。

---

3. 意图控制策略

> 评分表原文：这一项「评的是**你**，不是产品」。以下是被实际执行的、可复核的方法。

3.1 核心方法：给证伪任务，不给背书任务

这是整个项目里最关键的一次提问。

调研材料收齐后，我没有问”这些数据怎么支持我的选题”——那样 AI 一定会找到支持。 我问的是：

> **“这批数据支持不支持我们原来的定位？哪里对不上？”**

AI 找出了两处硬冲突：

1. 痛点集中在转专业**前**，而原定位是转专业**后**的补修路径规划
2. 笔试卷的是**高中物理与高等数学**，跟”目标专业的培养方案”没关系—— 这等于当面否掉了”工具能帮学生转专业”这个隐含承诺

原卖点”会不会延毕”被访谈直接否掉（“都不太在意延毕”）。

结果：核心能力没有改，但叙事入口从一个变成两个，并划出了明确的边界。

3.2 主动要求 AI 指出产品做不到什么

紧接着我要求 AI **主动划出”我们帮不上什么”**，并写进文档：

学生遇到的坎	我们能帮吗	依据
绩点 / 挂科门槛	❌ 学业硬指标	校方文件
转专业笔试	❌ 考的是高中物理与高数，与培养方案无关	校方文件
面试”对专业的了解”	⚠️ 间接支持	培养方案可视化能让学生看懂目标专业学什么；**不提供面试辅导**
查清准入规则	✅ 已实现	规则模块（`/api/rules`），且显式标注通知回答不了的部分
转专业后的**补修 / 重修清单**	✅ 已实现	只需培养方案 + 成绩单，不依赖认定办法（`/api/plan/credit-gap`）
转专业后的**学分折抵（旧课顶新课）**	❌ 暂不支持	依赖尚未获取的《学分认定管理办法》，且各校标准完全不同（见 6.2）

> 课径帮的是「同专业内的学分核算」，不是「跨专业的学分折抵」—— 前者有培养方案就能算，后者需要学校的认定规则。这两件事很容易被混为一谈， 所以在文档里分开写。

与其答辩时被评委戳破，不如自己先划清。 这一章后来成了文档里最受用的部分之一。

3.3 把契约前置到文档，让 AI 照契约写

在让 AI 写前端之前，我先把后端 9 个接口的**精确响应结构**（含嵌套字段名） 写进了规划文件，并明确”TS 类型照此写，字段名必须逐字符对上”。

这不是为了让 AI”知道得多”，而是为了**把一类错误提前消灭**：后端所有请求模型都是 `extra="forbid"`，把 `course_code` 写成 `courseCode` 会被 422。 与其等运行时发现，不如让契约先于代码存在。

3.4 要求可执行验收，而不是”完成了”

每一轮我都要求 AI 给出**可直接运行的验证命令**，并且自己跑一遍。这条纪律直接产出了 后面 9 个缺陷中的大多数——它们全都是”跑起来之后”才暴露的，看代码是看不出来的。

3.5 划出”AI 不许编”的红线

明确禁止两件事：

1. **不伪造校方数据**（排课时段、折抵规则）
2. **样例数据必须显式标注**（`data_status: "sample"` + `note` 说明边界与替换方式）

第一条的理由：项目 25% 的分数建立在”评委可一键复现”上，而伪造的数据不可复现。 第二条的理由：**样例与真实数据混淆，比没有数据更糟。**

这条红线在规则模块上被真正检验了一次：需求里点名要”分数门槛、笔试面试、专业和校区限制” 四类，把通知读完发现**其中三类根本不在文件里。**当时的选项是联网凑、填示例数据、 或者如实标注——最终没有编：该类别不放条款，改为列缺口 + 反向断言， 并写清”该去哪里查”。**红线只有在真的想编的时候才算红线。**

3.6 让 AI 换验证手段，而不是更仔细地看

第 5.2 节的第 3、4 号缺陷有个共同特征：**程序不报错、页面不异常，功能却缺了一块。** 静态截图验证不了”某个东西**没有**出现”。

于是我在例行验证之外让 AI 装了浏览器驱动（`playwright-core` + 系统已装的 Edge， 不下载浏览器），**真的去拖了一门课**，并收集控制台输出。一条命令同时验证了四件事：

验证项	实测结果
拖拽是否可用	✅ CS101 从第 1 学期拖到第 3 学期成功
冲突是否实时出现	✅ 出现 2 条红色冲突
点击能否联动高亮	✅ 高亮 2 张课程卡
流式是否真在逐字增长	✅ 首字 51ms，字数 73 → 107 持续增长
控制台错误	✅ 抓出第 4 号缺陷

---

4. 可复现性论证

4.1 一键复现路径

```
# 环境：Python 3.11+ / Node 18+（开发机实测 Node 24.15、npm 11.12）

# ① 后端依赖
py -3.11 -m venv .venv
.venv/Scripts/activate
pip install -r backend/requirements.txt

# ② 数据自检 —— 纯标准库，不装依赖也能跑
py -3.11 tools/validate_programs_json.py
#   期望：[OK] 样例-计算机科学与技术.json 41 门课 / 4 类学分要求 / data_status=sample
#         [√] 1 个培养方案全部通过校验

py -3.11 tools/validate_rules_json.py
#   期望：[OK] 转专业通知-枣庄学院-2024级.json（transfer-notice-2024）
#              17 条条款 / 5 个时间节点 / 5 处缺口 / data_status=official
#         [√] 1 份规则全部通过校验（引用与缺口断言均已回原文复核）

# ③ 后端单测
cd backend
pytest -q                      # 期望 159 passed
cd ..

# ④ 起后端（工作目录必须是 backend/）
cd backend
py -3.11 -m uvicorn app.main:app --reload --port 8000

# ⑤ 另开一个终端：前端
cd frontend
npm install
npm run typecheck && npm test  # 期望 typecheck 无输出、67 passed
npm run dev                    # → http://localhost:5173
```

全程无需任何 API Key，应用默认运行在 mock 模式。

4.2 数据管道的可复现

官方文件 → 结构化数据的每一步都有脚本，且脚本只依赖标准库：

产物	生成脚本	溯源信息
`data/majors-枣庄学院-2024级.json`	`tools/build_majors_json.py`	文件内 `source` 块记录了学校、原始 xlsx 路径、生成脚本
`data/rules/*.json`	`tools/build_rules_json.py`	同上 + **每条条款带原文条号与逐字引用**，自检脚本回原文复核
`data/programs/*.json`	无原件（样例数据）	`data_status: "sample"` + `note` 说明来源与替换方式

任何人拿到同一份官方 xlsx，运行同一个脚本，得到逐字节相同的 JSON。

规则数据把这条承诺推得更远一步。通知是散文而非表格，哪一段属于”挂科要求”、 哪一段属于”专业限制”正则判不出来，只能人工判读——**但判读结果可以被机器复核**： 每条条款的逐字引用都必须能在原文里找到，每个”通知没写分数线”的负向断言 都要经得起反向检索。判定权仍在人，复核权交给了脚本。

4.3 独立复现链

因为引擎层与数据层零第三方依赖，存在一条**不经过 web 栈**的复现路径：

```
py -3.11 tools/validate_programs_json.py
py -3.11 tools/validate_rules_json.py
```

有 Python 即可运行。这条链是”分层铁律”的直接产出，也是本项目与 “装完一堆依赖才能验证”的做法之间最实质的差别。

4.4 确定性

层面	保证方式	验证
冲突列表	显式 `KIND_ORDER` + 内部排序	打乱输入顺序，输出逐字段相等
mock 输出	关键词分派表，无随机、无时间戳	两次调用结果相等
流式分块	固定字数切分，延迟不进断言	`stream_delay=0` 时两次结果相等
规划结果	遍历按 `(term, course_code)` 排序	两次调用结果相等

4.5 流式的可证伪

“这是真流式”不靠嘴说，有四层证据，按说服力排序：

1. **命令行（完全绕开前端，最有说服力）**：

```
curl -N -X POST http://127.0.0.1:8000/api/llm/explain/stream \
  -H "Content-Type: application/json" \
  -d '{"program_id":"computer-science-2024-sample","items":
        [{"course_code":"CS101","term":3}],"completed":[]}'
# -N 关缓冲，肉眼可见 meta → thinking → delta… 逐条到达
```

2. **DevTools → Network → EventStream**：每帧带浏览器给的到达时间戳，做不了假。
3. **面板上的「停止生成」能真的中断**（`AbortController`）。
4. **单测断言同源性**：`"".join(stream(prompt)) == complete(prompt)`——流式不是第二份文案。

第 4 条是设计上的关键：`MockProvider._chunks()` 是**唯一的真相来源**，`complete()` 是它的拼接、`stream()` 是它加节奏。所以”流式”与”非流式”不可能不一致。

---

5. 工程质量

5.1 测试

层	数量	覆盖
后端（约束引擎 / 数据 / 接口 / 流式 / 规则 / 补修）	159	6 个检查函数的全部边界、13 个接口、SSE 帧完整性与同源性、规则引用逐字回溯、五类成绩单分类与学分守恒
前端（reducer / SSE 解帧 / 本地预判 / 规则登记表 / 成绩单）	82	原子变更、撤销重做、半帧跨块、CRLF、坏帧容错、类别分组、及格线边界、同名课录入

边界值被逐条钉死（举其要者）：学分/类别**恰好等于**下限 = 通过；上限为 0 = 不设限；先修成环用**显式栈 DFS**（递归实现遇 A↔B 会爆栈）；环内**不再逐边报告缺先修**（刻意抑制，有专门测试固定）；未知课程在规划里是 `error`、在成绩单里是 `info`（转专业学生的成绩单本来就含原专业课程）。

5.2 开发过程中修复的缺陷（10 个）

这一节是”代码非拼接”的实证：以下缺陷全部是**在本次开发中真实出现并被定位的**。

#	缺陷	性质	如何发现
1	`planner` 报的失败原因是错的（把”学分上限排不下”说成”先修时间冲突”）	逻辑错误	单测断言”提示应含『上限』“，测试红了
2	README 复现指南前后矛盾（71 行要求 CWD=仓库根，73 行要求 CWD=`backend/`）	**文档与实现脱节**	让 AI 把复现指南当成要执行的东西逐行读
3	`.env` 从未被加载（README 承诺”填 Key 后重启即可”，但 uvicorn 不读 `.env`）	**文档与实现脱节**	同上
4	专业列表把三种学制误标为”已接入培养方案”	匹配逻辑过宽	看截图
5	图表图例混入透明占位系列的名字	视觉噪声	看截图
6	上限线落在坐标区外被静默裁掉（y 轴自动收到 25，线在 26）	**静默失效**	看截图
7	ECharts 未注册 `LineChart`，挂上限线的系列被静默丢弃	**静默失效**	**浏览器控制台错误**
8	规则条款的引用转写错了（原文是「已修课程学习情况」，写成了「已修读课程学习情况」）	**引用不实**	引用回溯校验第一次运行就拒绝生成
9	顶栏 tab 的选中态被 `:hover` 盖住（伪类特异性 0,2,0 压过单类名 0,1,0）	样式覆盖 / **静默失效**	读 `getComputedStyle` 的实际背景色——**截图里看不出来**
10	跨专业同名课被误判为「需补修」：只按课号匹配，公共课在原专业有另一套课号	**计算错误**	预提交评审意见 → 写成 `test_same_name_same_credits_different_code_is_overlap` 复现

注意 3、7、9 的共同点：程序不报错、页面不异常，功能却缺了一块。 它们不会让演示崩掉，只会让演示”少一个东西”——而这正是最难发现的。

第 8 号是另一种典型：**它不是代码 bug，是我们自己转写时的记忆错误。** 值得注意的是 `docs/校方文件分析.md` 里同一句也是这么摘的——两处人工摘录犯了同一个错， 因此**拿产物去对照分析文档是发现不了的**，只有回原文才抓得住。 这也正是”逐字引用必须能被脚本复核”这条设计的由来。

第 2、3 号尤其值得说：它们不是算法错误，而是**承诺与实现脱节。** 程序不会因为它们崩掉，但”照着文档走的人”会卡住——而 25% 的分数恰恰压在 “评委能不能一键复现”上。发现它们的方法也很简单：**把文档当成要执行的东西读一遍。**

5.3 性能与体积取舍

项	实测	处理
前端首屏 JS	101.5 kB (gzip)	—
ECharts	178.9 kB (gzip)	按需引入 + **懒加载**（图表在侧栏偏下，不阻塞首屏）
后端校验	41 门课的 plan，请求体 < 4 KB，纯函数计算	不需要 WebSocket / 增量 diff

诚实说明：ECharts 的实际体积（178.9 kB gzip）是设计阶段估算的两倍。 它被单独切块并懒加载，因此不影响首屏；但如果认为”为一张堆叠柱不值这个体积”， 可以替换为约 60 行的内联 SVG 手绘（零依赖），界面质量不会下降。这是本项目 **第一个可以砍的技术债。**

---

6. 已知局限与边界

> **主动写下这一章，而不是等评委指出。** 本项目的取向是”宁可漏报，不要静默”， 对文档同样适用。

6.1 证据链的局限

1. **一手访谈缺失。** 原计划的”访谈 2–3 位转专业同学”未能执行（无可接触的转专业学生）。 证据链改由公开社区数据承担，但**“转专业后”环节的学生侧证据仍然薄弱**—— 只有一句汇总转述，没有原话、没有人次、没有耗时数据。
2. **社区样本量偏小**（3 条），扩采方案见 `docs/社区数据采集方案.md`（目标 30 条），尚未执行。
3. **幸存者偏差。** 二手报道的受访者 01 亲口点破： “这些人都是诸多转换专业道路上的幸存者”。 **这条警告的适用对象是我们自己**——社区帖里”难度并不高”的声音同样来自幸存者。
4. **时间跨度大。** 信源跨 2021–2026，转专业政策逐年在改，跨年度原话不能直接当作当前事实。

6.2 产品能力的边界

做不到	原因
转专业笔试辅导	笔试考高中物理与高数，与培养方案无关
绩点门槛相关建议	学业硬指标，非信息问题
跨专业学分折抵（旧课顶新课）	依赖尚未获取的《学校学分认定管理办法》。原专业课程目前归入 `pending`「待认定」，**不实现折算逻辑**
补修 / 重修清单	✅ 不在边界内——只需培养方案 + 成绩单，已实现（`/api/plan/credit-gap`）
时间冲突的完整检测	逻辑已实现并有单测覆盖，但缺排课时段数据（来源是排课表而非培养方案），样例数据不含该字段
跨校区转的限制	校方通知未覆盖，需向教务处确认

一句话边界：课径处理的是信息问题，不处理学业能力问题。

6.3 数据的边界

- `data/programs/` 下是**明确标注的样例数据**：专业名/学院/校区取自真实官方目录， 但课程、学分、先修关系为演示需要手工编写，**并非枣庄学院官方培养方案。**
- 学分量按演示需要缩放过（门槛 119 而非通常的 160+）。
- 「转专业规则查询」的**校级通知部分**已结构化并接入前端（`data/rules/`，17 条条款 / 5 个时间节点 / 5 处缺口）。21 个学院的《转专业工作方案》尚未采集——它们承载的 分数门槛、考核具体形式、校区限制三类规则，在接口里以 `gaps` 显式返回， 并附上反向断言（`absent_keywords`）说明"判定为没有"的依据，不做臆测填充。
- 「补修 / 重修清单」已实现（`/api/plan/credit-gap`）：课程号能对上的部分算得出来； 原专业课程归入「待认定」。**缺的是枣庄学院的《学分认定管理办法》，不是算法。** 跨校对照（上海交大给换算式、辽宁石化要求"课程目标等效"）见 `docs/校方文件分析.md` 第八节。

6.4 对预提交评审意见的回应

预提交评审提出：**"若因产品原因、功能不完善导致学生未能取得学分，该怎么办？"** 我们把这个问题拆成四层，逐层处理；完整回复见 `docs/评审意见回复.md`。

层	我们做了什么
计算	修掉”跨专业同名课被误判为需补修”的缺陷；新增「可能可折抵」第五类；前四类学分守恒写成单测断言
数据	示例成绩单从 4 门换为 33 门（软件工程 → 计算机），需补修 38 → 13，四类合计严格等于 123 学分；界面新增**数据覆盖度面板**（培养方案 1/108、学院方案 0/21、时段与认定办法未接入），把”缺什么”写在明面上
输出	五类分区 + 置信度 + 每类的动作；「可能可折抵」**不并进已认定**，明确”确认前不要当成已修”
责任	**可打印核对单已实现**：`POST /api/plan/audit-sheet` 输出五类清单 + 自检 + 三栏签字；样例数据下 `printable=false`，打印按钮置灰。纠错入口尚未实现

这条意见最有价值的地方，是逼我们把”不完善”从**缺陷**变成**显式功能**： 输出从”一个确定的数字”降级为”一份写明哪几条确定、哪几条需要人去确认的核对草案” 。

---

7. 附录

7.1 冲突类型（16 种）

按报告优先级排列，分属 6 个检查函数：

```
结构性  program_mismatch · unknown_course · duplicate_course · unknown_term
        unknown_completed_course · grade_unlabeled
先修    prerequisite_cycle · prerequisite_missing · prerequisite_order · prerequisite_failed
开课    course_not_offered · term_overload · time_overlap
学分    unknown_category · credit_below_minimum
边界    time_data_missing（不参与计数，单独区呈现）
```

7.2 开发过程留痕

- `docs/prompts/` —— 由 `tools/export_prompts.py` 从开发会话自动导出的完整 Prompt 链
- `docs/开发日志.md` —— 每轮迭代的人工复盘，重点是”遇到什么问题、怎么引导 AI 解决”
- Git 提交历史 —— 按功能模块切分，每条提交对应一轮 AI 对话

本文档本身就是可复现的：它由 `docs/技术文档.md` 经 `tools/build_tech_pdf.py` 生成 （pandoc → 独立 HTML → 无头 Edge 打印为 PDF），改完 Markdown 重跑一次脚本即可：

```
py -3.11 tools/build_tech_pdf.py      # → docs/技术文档.pdf，并报告页数
```

7.3 关键文件索引

路径	内容
`docs/用户调研.md`	学生侧证据：社区原话、二手报道、6 条发现、方法论局限
`docs/校方文件分析.md`	制度侧证据：官方规则还原、文件依赖图、交叉验证表
`docs/社区数据采集方案.md`	可复现的调研方法（含关键词清单与编码表）
`backend/app/constraints.py`	约束引擎（本项目技术核心）
`frontend/src/api/stream.ts`	SSE 解帧（全前端最易写错处，有 8 项边界测试）
`tools/validate_programs_json.py`	零依赖数据自检
`tools/validate_rules_json.py`	转专业规则自检：引用回溯 + 缺口反证
`backend/app/models.py` · `RuleDoc`	规则领域模型（条款 / 时间线 / 缺口）
`frontend/src/components/RulesView.tsx`	认知入口页面（条款 / 缺口 / 时间线 / 溯源）
`frontend/src/lib/ruleMeta.ts`	规则类别、判定与展示派生（纯函数，有单测）
`backend/app/planner.py` · `analyze_credits`	补修/重修清单的五类分类与学分守恒（纯函数）
`frontend/src/components/CreditGapView.tsx`	补修核算视图（成绩单录入 + 五类清单）